§.基础知识题 – C++方式输入输出的格式化控制



要求：

1、完成本文档中所有的题目并写出分析、运行结果

2、无特殊说明，均使用VS2022编译即可

3、直接在本文件上作答，写出答案/截图（不允许手写、手写拍照截图）即可；填写答案时，为适应所填内容或贴图，允许调整页面的字体大小、颜色、文本框的位置等

★ 贴图要有效部分即可，不需要全部内容

★ 在保证一页一题的前提下，具体页面布局可以自行发挥，简单易读即可

★ 不允许手写在纸上，再拍照贴图

★ 允许在各种软件工具上完成（不含手写），再截图贴图

★ 如果某题要求VS+Dev的，则如果两个编译器运行结果一致，贴VS的一张图即可，如果不一致，则两个图都要贴

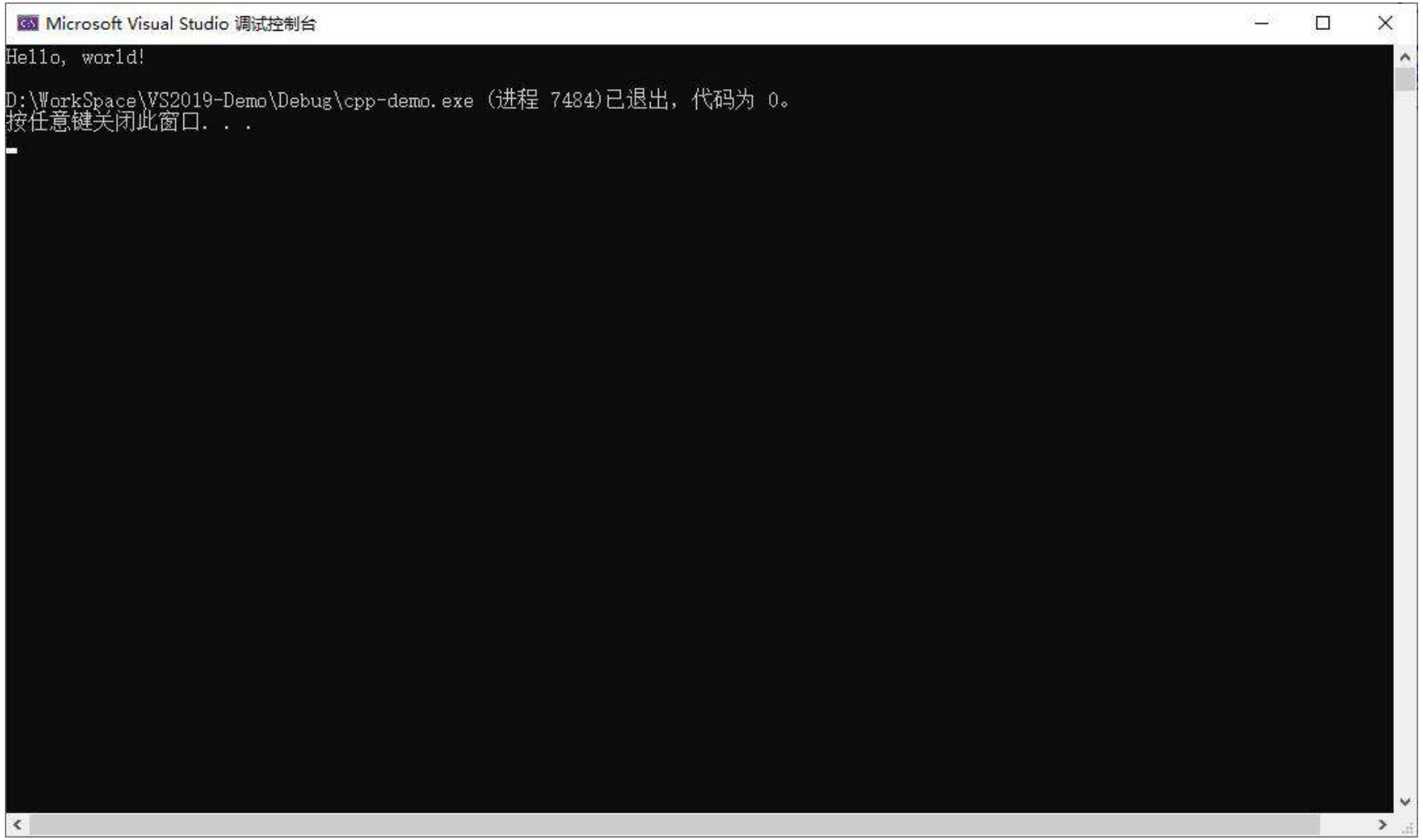
4、转换为pdf后提交

5、3月21日前网上提交本次作业（在“文档作业”中提交）

§.基础知识题 - C++方式输入输出的格式化控制

贴图要求：只需要截取输出窗口中的有效部分即可，如果全部截取/截取过大，则视为无效贴图

例：无效贴图

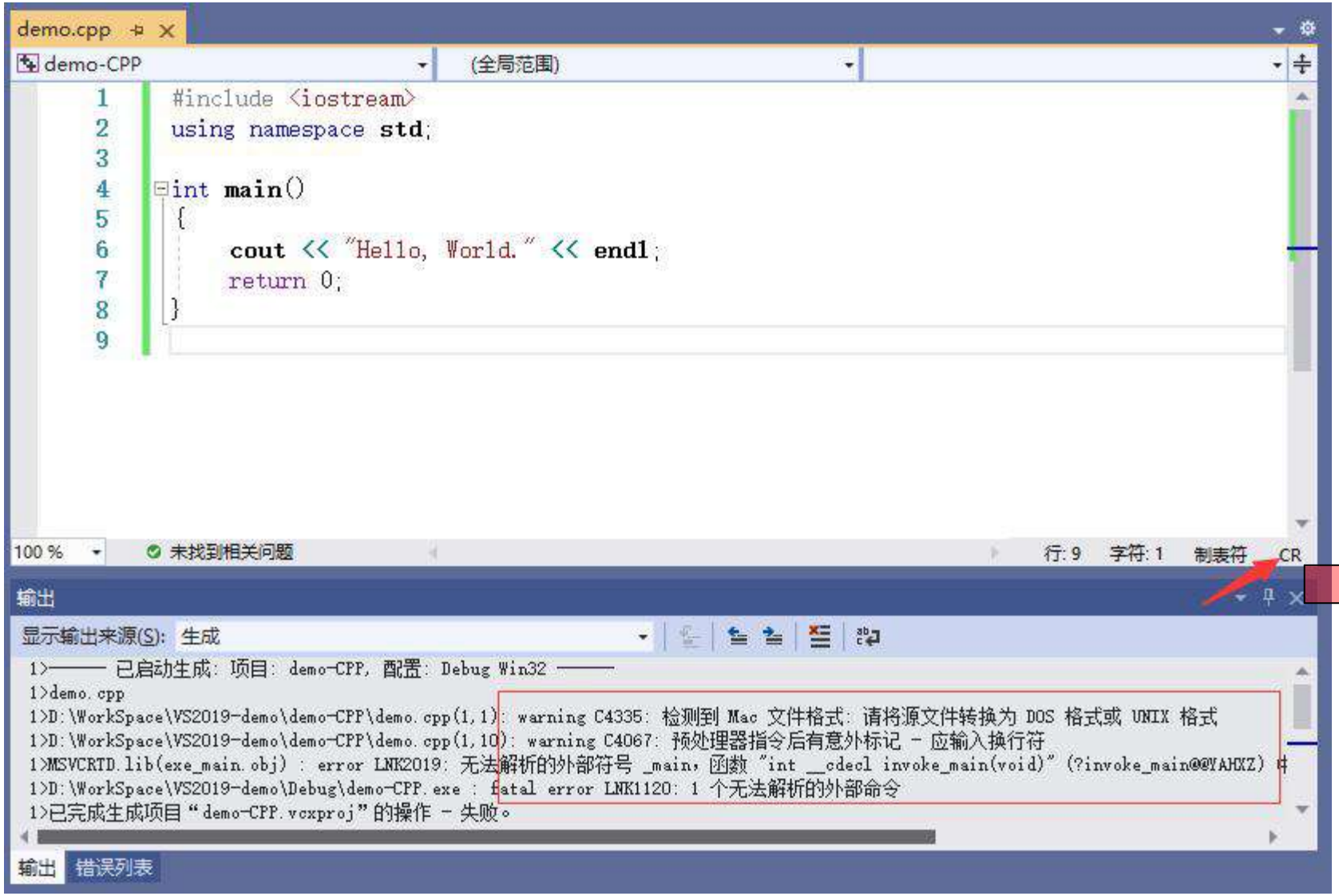


例：有效贴图



§.基础知识题 – C++方式输入输出的格式化控制

附：用WPS等其他第三方软件打开PPT，将代码复制到VS2022中后，如果出现类似下面的编译报错，则观察源程序编辑窗的右下角是否为CR，如果是，单击CR，在弹出中选择CRLF，再次CTRL+F5运行即可





§.基础知识题 – C++方式输入输出的格式化控制

特别提示：

1、做题过程中，先按要求输入，如果想替换数据，也要先做完指定输入

2、如果替换数据后出现某些问题，先记录下来，不要问，等全部完成后，还想不通再问(也许你的问题在后面的题目中有答案)

3、不要偷懒、不要自以为是的脑补结论!!!

4、先得到题目要求的小结论，再综合考虑上下题目间关系，得到综合结论

5、这些结论，是让你记住的，不是让你完成作业后就忘掉了

6、换位思考(从老师角度出发)，这些题的目的是希望掌握什么学习方法？

§.基础知识题 – C++方式输入输出的格式化控制



说明：C++中的格式控制很丰富，实现方法也有多种，下表列出的只是常用一部分，用于本次作业

控制符	作用
dec	设置整数为10进制
hex	设置整数为16进制
oct	设置整数为8进制
setbase(n)	设置整数为n进制(n=8,10,16)
setfill(c)	设置填充字符，c可以是字符常量或字符变量
setprecision(n)	设置实数的精度为n位。在以一般十进制形式输出时，n代表有效数字。 在以fixed(固定小数位)形式和scientific(指数)形式输出时，n为小数位数
setw(n)	设置字段宽度为n
setiosflags(ios::fixed)	设置浮点数以固定的小数位数显示
setiosflags(ios::scientific)	设置浮点数以科学计数法（即指数形式）显示
setiosflags(ios::left)	输出数据左对齐
setiosflags(ios::right)	输出数据右对齐
setiosflags(ios::skipws)	忽略前导的空格
setiosflags(ios::uppercase)	在以科学计数法输出E和十六进制输出字母X时，以大写表示
setiosflags(ios::showpos)	输出正数时，给出“+”号
resetiosflags(*)	终止已设置的输出格式状态，括号内为具体内容(本处用*替代)

重要提示：

1、后面作业需要的知识点，除非明确提示自行上网查找，都先在本文档中查找是否有符合要求的设置项

2、不看本页，网上瞎找，然后说作业多的，本课程及本作业不背锅

§.基础知识题 – C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

A.进制前导符的使用：回答问题并将程序的运行结果截图贴上(允许多页)

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    short a1 = 1234, a2 = 0x1234, a3 = 01234, a4 = 0b1101001;  //常量为各进制表示正数
    cout << "dec:" << dec << a1 << ' ' << a2 << ' ' << a3 << ' ' << a4 << endl;
    cout << "hex:" << hex << a1 << ' ' << a2 << ' ' << a3 << ' ' << a4 << endl;
    cout << "oct:" << oct << a1 << ' ' << a2 << ' ' << a3 << ' ' << a4 << endl;
    cout << endl;

    short b1 = -1234, b2 = -0x1234, b3 = -01234, b4 = -0b1101001;  //常量为各进制表示负数
    cout << "dec:" << dec << b1 << ' ' << b2 << ' ' << b3 << ' ' << b4 << endl;
    cout << "hex:" << hex << b1 << ' ' << b2 << ' ' << b3 << ' ' << b4 << endl;
    cout << "oct:" << oct << b1 << ' ' << b2 << ' ' << b3 << ' ' << b4 << endl;
    cout << endl;

    short c1 = 40000, c2 = 0x9876, c3 = 0171234, c4 = 0b1101010100111100;  //赋值后最高位均为1，有warning
    cout << "dec:" << dec << c1 << ' ' << c2 << ' ' << c3 << ' ' << c4 << endl;
    cout << "hex:" << hex << c1 << ' ' << c2 << ' ' << c3 << ' ' << c4 << endl;
    cout << "oct:" << oct << c1 << ' ' << c2 << ' ' << c3 << ' ' << c4 << endl;
    cout << endl;

    return 0;
}

//允许贴图覆盖代码部分
```



§.基础知识题 – C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

A.总结及结论：

1、源程序中的整数，有___3___种不同进制的表示形式

2、无论源程序中整型常量表示为何种进制，它的机内存储均为___二进制_____形式

3、如果想使数据输出时使用不同进制，要加___dec、hex、oct_____等进制前导符

4、输出___无____(有/无)二进制前导符

5、只有____十____进制有负数形式输出；
16进制输出负数时，特征是___将该负数的补码视为一个二进制正数转变而来的十六进制数_____；
8进制输出负数时，特征是____将该负数的补码视为一个二进制正数转变而来的八进制数_____

§.基础知识题 – C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

B.进制前导符的连续使用：回答问题并将程序的运行结果截图贴上

```
#include <iostream>
#include <iomanip>

using namespace std;
int main()
{
    int a = 10;

    cout << a << ' ' << a+1 << ' ' << a+2 << endl;
    cout << hex;
    cout << a << ' ' << a+1 << ' ' << a+2 << endl;
    cout << oct;
    cout << a << ' ' << a+1 << ' ' << a+2 << endl;
    cout << dec;
    cout << a << ' ' << a+1 << ' ' << a+2 << endl;

    return 0;
}
```



结论：

dec/hex/oct等进制前导符设置后，对后面的___所有____(仅一个/所有)数据有效，直到用另一个控制符去改变为止

§.基础知识题 – C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

C.setbase的使用：同1.A的形式，按要求自行构造测试程序，回答问题并将程序的运行结果截图贴上(允许多页)

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    short a1 = 1234, a2 = 0x1234, a3 = 01234, a4 = 0b1101001;  //常量为各进制表示正数
    cout << "dec:" << setbase(10) << a1 << ' ' << a2 << ' ' << a3 << ' ' << a4 << endl;
    cout << "hex:" << setbase(16) << a1 << ' ' << a2 << ' ' << a3 << ' ' << a4 << endl;
    cout << "oct:" << setbase(8) << a1 << ' ' << a2 << ' ' << a3 << ' ' << a4 << endl;
    cout << endl;

    short b1 = -1234, b2 = -0x1234, b3 = -01234, b4 = -0b1101001;  //常量为各进制表示负数
    cout << "dec:" << setbase(10) << b1 << ' ' << b2 << ' ' << b3 << ' ' << b4 << endl;
    cout << "hex:" << setbase(16) << b1 << ' ' << b2 << ' ' << b3 << ' ' << b4 << endl;
    cout << "oct:" << setbase(8) << b1 << ' ' << b2 << ' ' << b3 << ' ' << b4 << endl;
    cout << endl;

    short c1 = 40000, c2 = 0x9876, c3 = 0171234, c4 = 0b1101010100111100;  //赋值后最高位
    cout << "dec:" << setbase(10) << c1 << ' ' << c2 << ' ' << c3 << ' ' << c4 << endl;
    cout << "hex:" << setbase(16) << c1 << ' ' << c2 << ' ' << c3 << ' ' << c4 << endl;
    cout << "oct:" << setbase(8) << c1 << ' ' << c2 << ' ' << c3 << ' ' << c4 << endl;
    cout << endl;

    return 0;
}
```

```
Microsoft Visual Studio 调试控制台
dec:1234 4660 668 105
hex:4d2 1234 29c 69
oct:2322 11064 1234 151

dec:-1234 -4660 -668 -105
hex:fb2e edcc fd64 ff97
oct:175456 166714 176544 177627

dec:-25536 -26506 -3428 -10948
hex:9c40 9876 f29c d53c
oct:116100 114166 171234 152474
```

未找到相关问题

生成

: 1 成功，0 失败，0 最新，

自行构造若干组测试数据，运行并截图

结论：

1、setbase中允许的合法值有__8,10,16_____

2、当setbase中出现非法值时，处理方法是____无视setbase，使用10进制输出_____

3、setbase设置后，对后面的___所有_____(仅一个/所有)数据

有效，直到用另一个setbase去改变为止

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    short a1 = 1234, a2 = 0x1234, a3 = 01234, a4 = 0b1101001;
    cout << setbase(2) << a1 << ' ' << a2 << ' ' << a3 << ' ' << a4 << endl;

    return 0;
}
```

```
Microsoft Visual Studio 调试控制台
1234 4660 668 105
```

//构造的程序要求能看出对右侧问题的回答

//将构造的程序直接贴图上来，左侧不写也可

§.基础知识题 - C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

D.ios::uppercase的使用：按要求自行构造测试程序，能对比看出用和不用的差别即可

```
#include <iostream>
#include <iomanip>

using namespace std;
int main()
{
    return 0;
}//允许直接贴构造的程序，不用再输入到这里
```

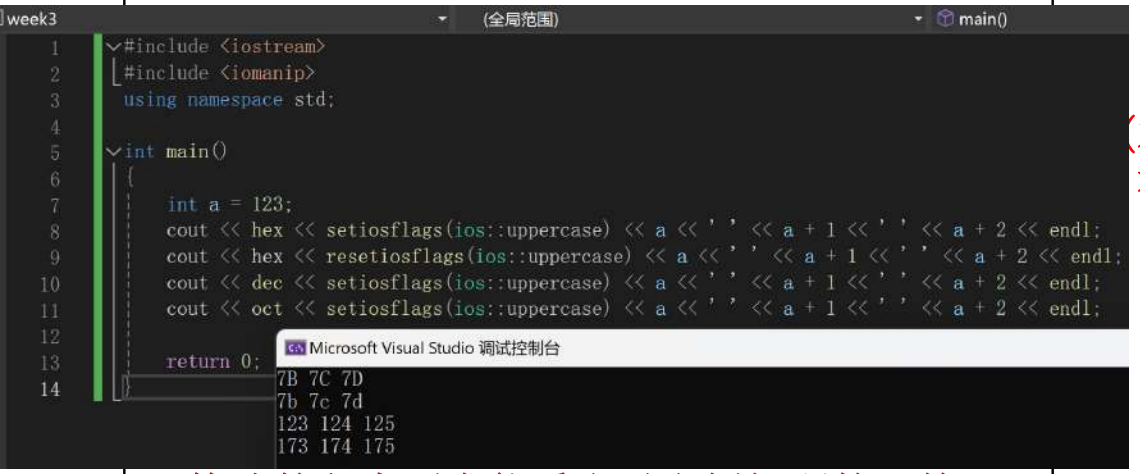
测试程序中的数据类型为int，自行构造若干组测试数据，运行并截图

结论：

1、uppercase和_十六__进制一起使用才能看出效果

2、uppercase设置后，对后面的___所有_____(仅一个/所有)数据

有效



同一个程序中，设置完uppercase，如果想恢复小写，具体的做法是___设置restiosflags(ios::uppercase)_____

(本小问如果不会，先不要问，先往后做，看后面的题目是否有相似问题可以启发你)

//构造的程序要求能看出对右侧问题的回答

//将构造的程序直接贴图上来，左侧不写也可

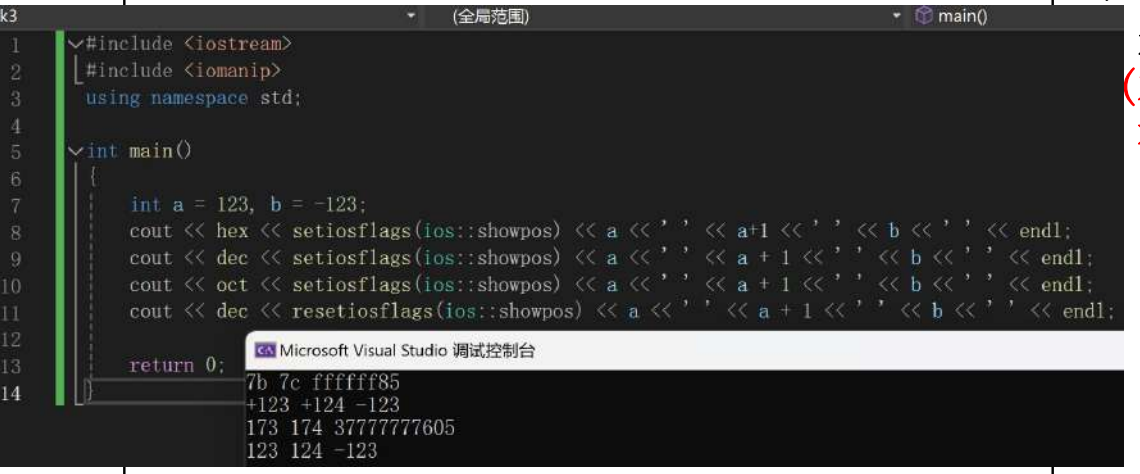
§.基础知识题 – C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

E.ios::showpos的使用：按要求自行构造测试程序，能对比看出用和不用的差别即可

```
#include <iostream>
#include <iomanip>

using namespace std;
int main()
{
    return 0;
}//允许直接贴构造的程序，不用再输入到这里
```



//构造的程序要求能看出对右侧问题的回答

//将构造的程序直接贴图上来，左侧不写也可

测试程序中的数据类型为int，自行构造若干组测试数据，运行并截图

结论：

1、showpos和__十_进制一起使用才能看出效果

2、showpos设置后，对后面的___所有___(仅一个/所有)数据有效

3、同一个程序中，设置完showpos，如果想取消，具体的做法是___设置restiosflags(ios:: showpos)_____

(本小问如果不会，先不要问，先往后做，看后面的题目是否有相似问题可以启发你)

§.基础知识题 - C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

F.setprecision的使用 - 单独使用 - (1)

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    float f1 = 1234.5678F;
    float f2 = 8765.4321F;

    /* 第1组：不设或非法 */
    cout << f1 << ' ' << f2 << endl;
    cout << setprecision(0) << f1 << ' ' << f2 << endl;

    /* 第2组：小于等于整数位数 */
    cout << endl;
    cout << setprecision(1) << f1 << ' ' << f2 << endl;
    cout << setprecision(2) << f1 << ' ' << f2 << endl;
    cout << setprecision(3) << f1 << ' ' << f2 << endl;
    cout << setprecision(4) << f1 << ' ' << f2 << endl;

    /* 第3组：大于整数位数，但小与等于float型有效数字 */
    cout << endl;
    cout << setprecision(5) << f1 << ' ' << f2 << endl;
    cout << setprecision(6) << f1 << ' ' << f2 << endl;
    cout << setprecision(7) << f1 << ' ' << f2 << endl;

    /* 第4组：大于float型有效数字 */
    cout << endl;
    cout << setprecision(8) << f1 << ' ' << f2 << endl;
    cout << setprecision(9) << f1 << ' ' << f2 << endl;
    cout << setprecision(10) << f1 << ' ' << f2 << endl;
    cout << setprecision(25) << f1 << ' ' << f2 << endl;

    return 0;
}
```

本例贴图



§.基础知识题 - C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

F.setprecision的使用 - 单独使用 - (2)

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    float f1 = 1234567890123456789.0F;
    float f2 = 9876543210987654321.0F;

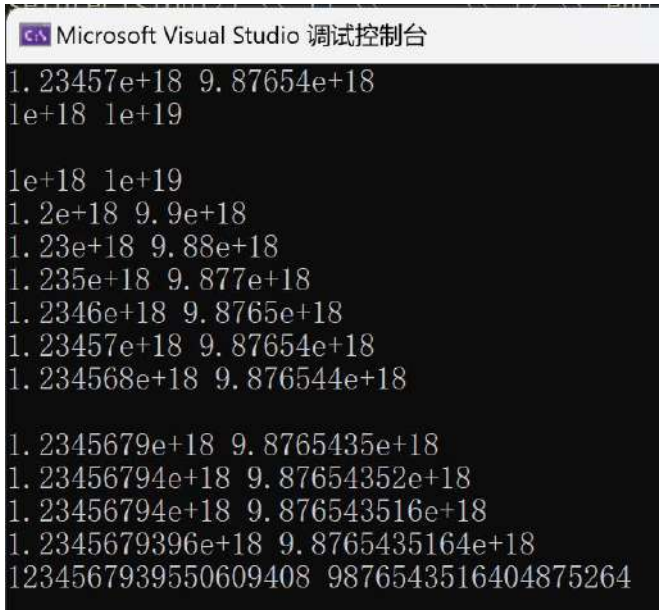
    /* 第1组：不设或非法 */
    cout << f1 << ' ' << f2 << endl;
    cout << setprecision(0) << f1 << ' ' << f2 << endl;

    /* 第2组：小于等于整数位数 并且 小与等于float型有效数字 */
    cout << endl;
    cout << setprecision(1) << f1 << ' ' << f2 << endl;
    cout << setprecision(2) << f1 << ' ' << f2 << endl;
    cout << setprecision(3) << f1 << ' ' << f2 << endl;
    cout << setprecision(4) << f1 << ' ' << f2 << endl;
    cout << setprecision(5) << f1 << ' ' << f2 << endl;
    cout << setprecision(6) << f1 << ' ' << f2 << endl;
    cout << setprecision(7) << f1 << ' ' << f2 << endl;

    /* 第3组：大于float型有效数字 */
    cout << endl;
    cout << setprecision(8) << f1 << ' ' << f2 << endl;
    cout << setprecision(9) << f1 << ' ' << f2 << endl;
    cout << setprecision(10) << f1 << ' ' << f2 << endl; //为什么f1比f2少一位？因为f1最后一位是0，省略（可由下一行“9396”进位成“940”推出）
    cout << setprecision(11) << f1 << ' ' << f2 << endl;
    cout << setprecision(25) << f1 << ' ' << f2 << endl;

    return 0;
}
```

本例贴图



§.基础知识题 - C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

F.setprecision的使用 - 单独使用 - (3)

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    float f1 = 0.12345678F;
    float f2 = 0.87654321F;

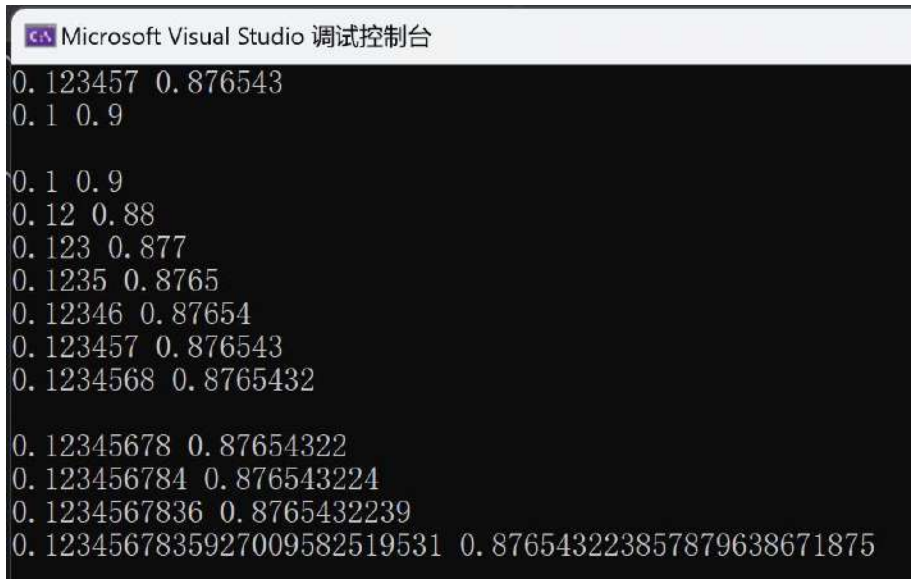
    /* 第1组：不设或非法 */
    cout << f1 << ' ' << f2 << endl;
    cout << setprecision(0) << f1 << ' ' << f2 << endl;

    /* 第2组：小与等于float型有效数字 */
    cout << endl;
    cout << setprecision(1) << f1 << ' ' << f2 << endl;
    cout << setprecision(2) << f1 << ' ' << f2 << endl;
    cout << setprecision(3) << f1 << ' ' << f2 << endl;
    cout << setprecision(4) << f1 << ' ' << f2 << endl;
    cout << setprecision(5) << f1 << ' ' << f2 << endl;
    cout << setprecision(6) << f1 << ' ' << f2 << endl;
    cout << setprecision(7) << f1 << ' ' << f2 << endl;

    /* 第3组：大于float型有效数字 */
    cout << endl;
    cout << setprecision(8) << f1 << ' ' << f2 << endl;
    cout << setprecision(9) << f1 << ' ' << f2 << endl;
    cout << setprecision(10) << f1 << ' ' << f2 << endl;
    cout << setprecision(25) << f1 << ' ' << f2 << endl;

    return 0;
}
```

本例贴图



§.基础知识题 - C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

F.setprecision的使用 - 单独使用 - 总结

重要结论：setprecision指定输出位数后，系统会按指定位数输出，即使指定位数超过数据的有效位数
（即：输出数据的某位开始是不可信的，但依然会输出）

1、给出setprecision单独使用时的显示规律总结（如果数据不够，可以再自己构造测试数据）

输出位数大于等于整数位数时，不使用科学计数法，小于整数位数时，使用科学计数法。进位时四舍五入。小数部分末尾的0会省略。不设置setpercision时，默认输出6位有效数字；非法设置0会输出1位有效数字，设置负数（无论整数小数）会输出6位有效数字，设置正小数会去尾法当做整数并报warning；设置小于float型的最小有效数字（6或7）时，会正确输出；设置等于float型的有效数字（6或7）时，可能会输出正确数也可能会输出错误数，错误数只会是最后一位，是否错误取决于后一位（7或8位）的进位；设置大于float型有效数字（6或7），会输出不可信数字，即使指定位数超过数据的有效位数，后面也会一直输出下去，绝大部分情况下，从float有效位数的后一位（第7或8位）开始不可信，极少数数字正好在float型下精度极高，则可能从很后面的位数才开始不可信，甚至没有不可信的位数。

week3 (全局范围)

```cpp
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    //设置负数
    float f = 1234.123f;
    cout << setprecision(-1) << f << endl;
    cout << setprecision(-2) << f << endl;
    cout << setprecision(-3) << f << endl;
    cout << setprecision(-6) << f << endl;
    cout << setprecision(-7) << f << endl;
    cout << setprecision(-10) << f << endl;
    cout << endl;
    //设置小数
    cout << setprecision(1.5) << f << endl;
    cout << setprecision(0.2) << f << endl;
    cout << setprecision(2.3) << f << endl;
    cout << setprecision(3.648) << f << endl;
    cout << setprecision(10.23) << f << endl;
    cout << setprecision(-1.5) << f << endl;
    cout << setprecision(-10.648) << f << endl;
    //精度极高的数字
    float f1 = 0.99999994039f;
    cout << setprecision(10) << f1 << endl;

    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
1234.12
1234.12
1234.12
1234.12
1234.12
1234.12

1e+03
1e+03
1.2e+03
1.23e+03
1234.123047
1234.12
1234.12
0.9999999404
```

```
warning C4244: “参数”：从“double”转换到“std::streamsize”，可能丢失数据
warning C4244: “参数”：从“double”转换到“std::streamsize”，可能丢失数据
warning C4244: “参数”：从“double”转换到“std::streamsize”，可能丢失数据
warning C4244: “参数”：从“double”转换到“std::streamsize”，可能丢失数据
warning C4244: “参数”：从“double”转换到“std::streamsize”，可能丢失数据
warning C4244: “参数”：从“double”转换到“std::streamsize”，可能丢失数据
warning C4244: “参数”：从“double”转换到“std::streamsize”，可能丢失数据
```

2、将1.F-(1)~(3)中的数据类型换为double型（有效位数为15位），自行构造测试数据，验证总结出的float型数据的显示规律是否同样适用于double型（如果适用，不用贴图，如果不适用，贴对应代码及运行截图）

§.基础知识题 - C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

G.setprecision的使用 - 和ios::fixed一起 - (1)

```
#include <iostream>
#include <iomanip>

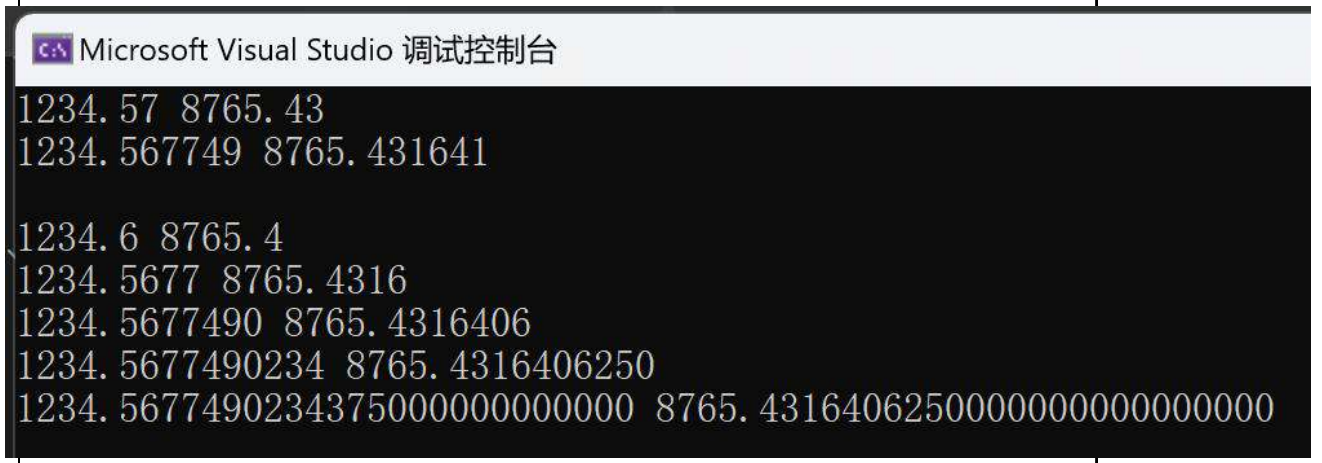
using namespace std;
int main()
{
    float f1 = 1234.5678F;
    float f2 = 8765.4321F;

    /* 第1组：不设precision */
    cout << f1 << ' ' << f2 << endl;
    cout << setiosflags(ios::fixed) << f1 << ' ' << f2 << endl;

    /* 第2组：设置precision */
    cout << endl;
    cout << setprecision(1) << f1 << ' ' << f2 << endl;
    cout << setprecision(4) << f1 << ' ' << f2 << endl;
    cout << setprecision(7) << f1 << ' ' << f2 << endl;
    cout << setprecision(10) << f1 << ' ' << f2 << endl;
    cout << setprecision(25) << f1 << ' ' << f2 << endl;

    return 0;
}
```

贴图：



§.基础知识题 - C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

G.setprecision的使用 - 和ios::fixed一起 - (2)

```
#include <iostream>
#include <iomanip>

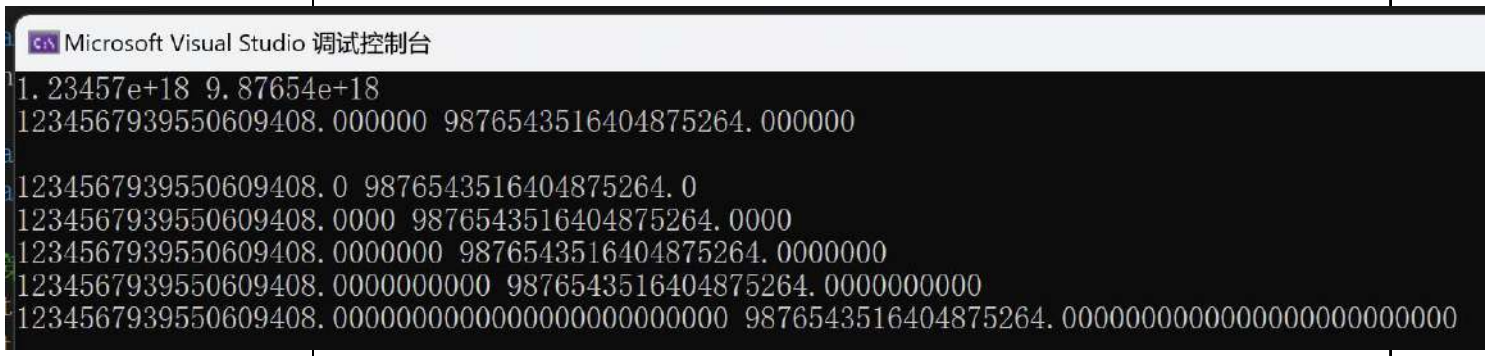
using namespace std;
int main()
{
    float f1 = 1234567890123456789.0F;
    float f2 = 9876543210987654321.0F;

    /* 第1组：不设precision */
    cout << f1 << ' ' << f2 << endl;
    cout << setiosflags(ios::fixed) << f1 << ' ' << f2 << endl;

    /* 第2组：设置precision */
    cout << endl;
    cout << setprecision(1) << f1 << ' ' << f2 << endl;
    cout << setprecision(4) << f1 << ' ' << f2 << endl;
    cout << setprecision(7) << f1 << ' ' << f2 << endl;
    cout << setprecision(10) << f1 << ' ' << f2 << endl;
    cout << setprecision(25) << f1 << ' ' << f2 << endl;

    return 0;
}
```

贴图：



§.基础知识题 - C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

G.setprecision的使用 - 和ios::fixed一起 - (3)

```
#include <iostream>
#include <iomanip>

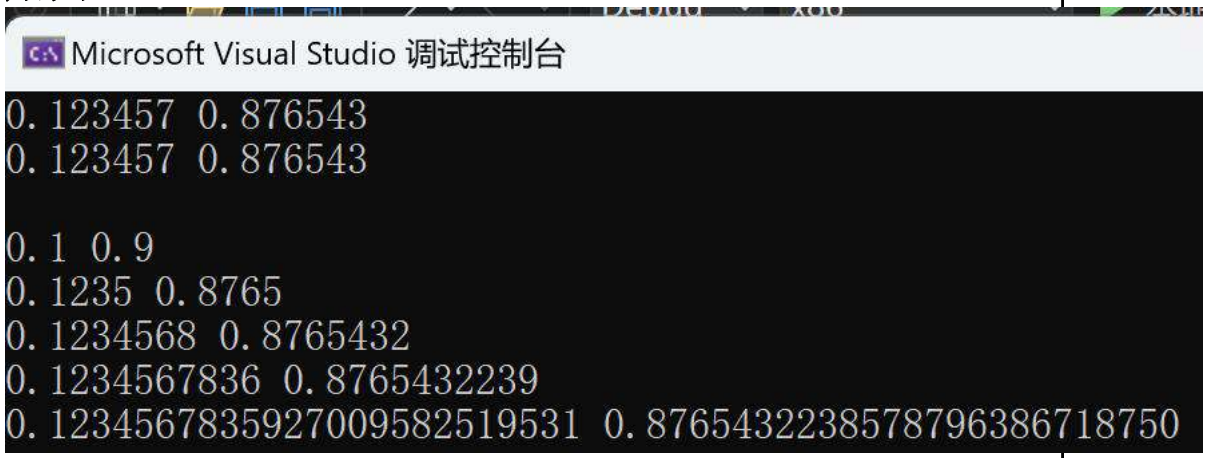
using namespace std;
int main()
{
    float f1 = 0.12345678F;
    float f2 = 0.87654321F;

    /* 第1组：不设precision */
    cout << f1 << ' ' << f2 << endl;
    cout << setiosflags(ios::fixed) << f1 << ' ' << f2 << endl;

    /* 第2组：设置precision */
    cout << endl;
    cout << setprecision(1) << f1 << ' ' << f2 << endl;
    cout << setprecision(4) << f1 << ' ' << f2 << endl;
    cout << setprecision(7) << f1 << ' ' << f2 << endl;
    cout << setprecision(10) << f1 << ' ' << f2 << endl;
    cout << setprecision(25) << f1 << ' ' << f2 << endl;

    return 0;
}
```

贴图：



§.基础知识题 - C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

G.setprecision的使用 - 和ios::fixed一起 - 总结

1、给出setprecision+ios::fixed使用时的显示规律总结（如果数据不够，可以再自己构造测试数据）
使用ios::fixed以后，默认输出六位小数，不使用科学计数法，小数点的位置和赋值时一样，小数部分末尾的0不再省略，数据的可信规则和之前总结的单独使用setprecision时一样； setprecision+ios::fixed一起使用，不使用科学计数法，小数点的位置和赋值时一样，小数部分末尾的0不再省略，输出setprecision设置的数字的小数位数，数据的可信规则、处理负数和小数的方式和之前总结的单独使用setprecision时一样

week3 (全局范围)

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    float f = 1234.123f;
    cout << setiosflags(ios::fixed) << f << endl;
    //设置负数
    cout << setprecision(-1) << f << endl;
    cout << setprecision(-2) << f << endl;
    cout << setprecision(-3) << f << endl;
    cout << setprecision(-6) << f << endl;
    cout << setprecision(-7) << f << endl;
    cout << setprecision(-10) << f << endl;
    cout << endl;
    //设置小数
    cout << setprecision(1.5) << f << endl;
    cout << setprecision(0.2) << f << endl;
    cout << setprecision(2.3) << f << endl;
    cout << setprecision(3.648) << f << endl;
    cout << setprecision(10.23) << f << endl;
    cout << setprecision(-1.5) << f << endl;
    cout << setprecision(-10.648) << f << endl;

    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
1234.123047
1234.123047
1234.123047
1234.123047
1234.123047
1234.123047
1234.123047

1234.1
1234
1234.12
1234.123
1234.1230468750
1234.123047
1234.123047
```

2、将1.G-(1)~(3)中的数据类型换为double型（有效位数为15位），自行构造测试数据，验证总结出的float型数据的显示规律是否同样适用于double型（如果适用，不用贴图，如果不适用，贴对应代码及运行截图）

§.基础知识题 - C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

H.setprecision的使用 - 和ios::scientific一起 - (1)

```
#include <iostream>
#include <iomanip>

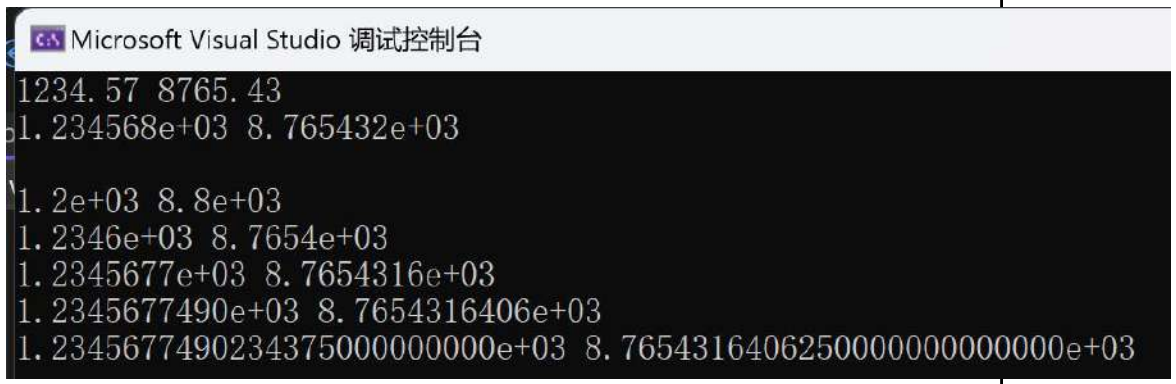
using namespace std;
int main()
{
    float f1 = 1234.5678F;
    float f2 = 8765.4321F;

    /* 第1组：不设precision */
    cout << f1 << ' ' << f2 << endl;
    cout << setiosflags(ios::scientific) << f1 << ' ' << f2 << endl;

    /* 第2组：设置precision */
    cout << endl;
    cout << setprecision(1) << f1 << ' ' << f2 << endl;
    cout << setprecision(4) << f1 << ' ' << f2 << endl;
    cout << setprecision(7) << f1 << ' ' << f2 << endl;
    cout << setprecision(10) << f1 << ' ' << f2 << endl;
    cout << setprecision(25) << f1 << ' ' << f2 << endl;

    return 0;
}
```

贴图：



§.基础知识题 - C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

H.setprecision的使用 - 和ios::scientific一起 - (2)

```
#include <iostream>
#include <iomanip>

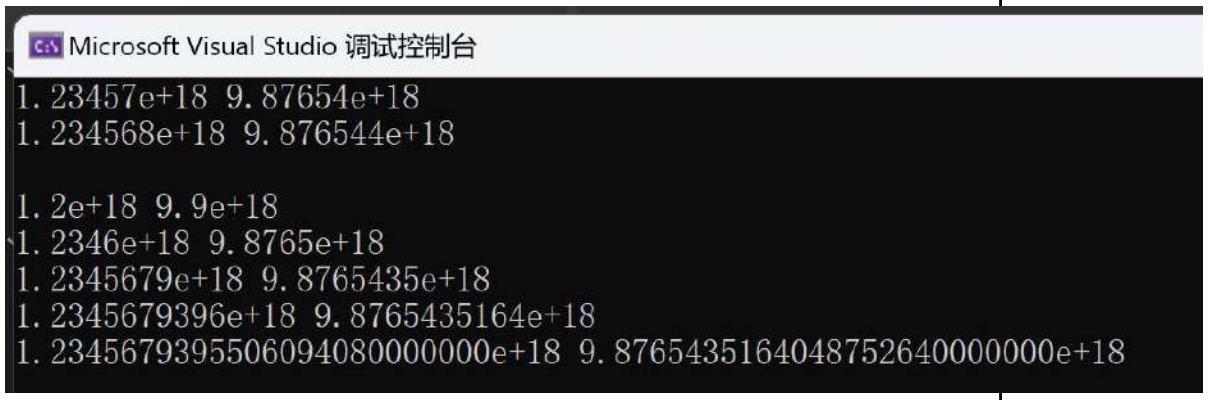
using namespace std;
int main()
{
    float f1 = 1234567890123456789.0F;
    float f2 = 9876543210987654321.0F;

    /* 第1组：不设precision */
    cout << f1 << ' ' << f2 << endl;
    cout << setiosflags(ios::scientific) << f1 << ' ' << f2 << endl;

    /* 第2组：设置precision */
    cout << endl;
    cout << setprecision(1) << f1 << ' ' << f2 << endl;
    cout << setprecision(4) << f1 << ' ' << f2 << endl;
    cout << setprecision(7) << f1 << ' ' << f2 << endl;
    cout << setprecision(10) << f1 << ' ' << f2 << endl;
    cout << setprecision(25) << f1 << ' ' << f2 << endl;

    return 0;
}
```

贴图：



§.基础知识题 - C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

H.setprecision的使用 - 和ios::scientific一起 - (3)

```
#include <iostream>
#include <iomanip>

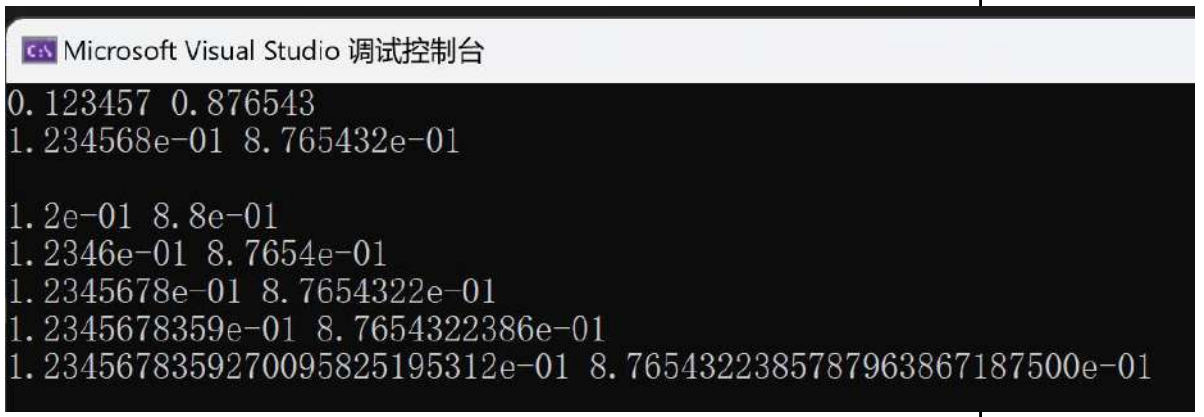
using namespace std;
int main()
{
    float f1 = 0.12345678F;
    float f2 = 0.87654321F;

    /* 第1组：不设precision */
    cout << f1 << ' ' << f2 << endl;
    cout << setiosflags(ios::scientific) << f1 << ' ' << f2 << endl;

    /* 第2组：设置precision */
    cout << endl;
    cout << setprecision(1) << f1 << ' ' << f2 << endl;
    cout << setprecision(4) << f1 << ' ' << f2 << endl;
    cout << setprecision(7) << f1 << ' ' << f2 << endl;
    cout << setprecision(10) << f1 << ' ' << f2 << endl;
    cout << setprecision(25) << f1 << ' ' << f2 << endl;

    return 0;
}
```

贴图：



§.基础知识题 - C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

H.setprecision的使用 - 和ios::scientific一起 - 总结

1、给出setprecision+ios::scientific使用时的显示规律总结（如果数据不够，可以再自己构造测试数据）
使用ios::scientific后，使用科学计数法输出，输出6位小数，小数末尾的0不再省略，数据的可信规则和之前总结的单独使用setprecision时一样； setprecision+ios::scientific一起使用，使用科学计数法输出且小数位数和setpercision设置的数字一样，小数末尾的0不再省略，数据的可信规则、处理负数和小数的方式和之前总结的单独使用setprecision时一样

week3 (全局范围)

```cpp
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    float f = 1234.123f;
    cout << setiosflags(ios::scientific) << f << endl;
    //设置负数
    cout << setprecision(-1) << f << endl;
    cout << setprecision(-2) << f << endl;
    cout << setprecision(-3) << f << endl;
    cout << setprecision(-6) << f << endl;
    cout << setprecision(-7) << f << endl;
    cout << setprecision(-10) << f << endl;
    cout << endl;
    //设置小数
    cout << setprecision(1.5) << f << endl;
    cout << setprecision(0.2) << f << endl;
    cout << setprecision(2.3) << f << endl;
    cout << setprecision(3.648) << f << endl;
    cout << setprecision(10.23) << f << endl;
    cout << setprecision(-1.5) << f << endl;
    cout << setprecision(-10.648) << f << endl;

    return 0;
```

Microsoft Visual Studio 调试控制台

```
1.234123e+03
1.234123e+03
1.234123e+03
1.234123e+03
1.234123e+03
1.234123e+03
1.234123e+03

1.2e+03
1e+03
1.23e+03
1.234e+03
1.2341230469e+03
1.234123e+03
1.234123e+03
```

2、将1.H-(1)~(3)中的数据类型换为double型（有效位数为15位），自行构造测试数据，验证总结出的float型数据的显示规律是否同样适用于double型（如果适用，不用贴图，如果不适用，贴对应代码及运行截图）

§.基础知识题 – C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

I.ios::fixed和ios::scientific的混合使用 – 错误用法

```
#include <iostream>
#include <iomanip>

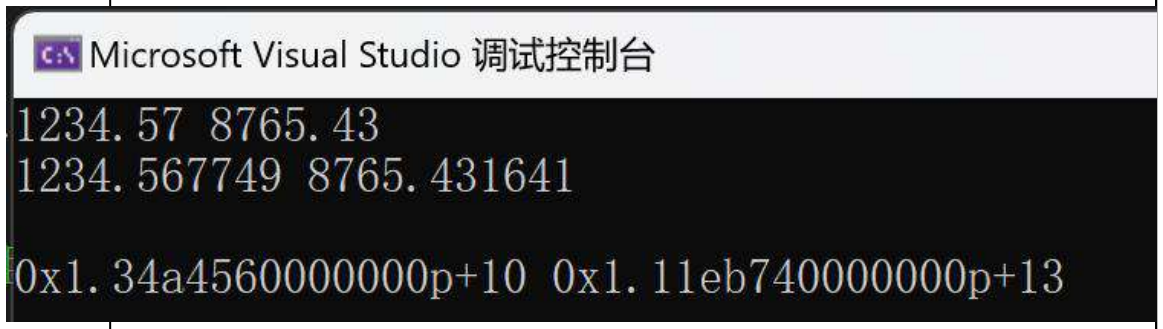
using namespace std;
int main()
{
    float f1 = 1234.5678F, f2 = 8765.4321F;

    /* 第1组 */
    cout << f1 << ' ' << f2 << endl;
    cout << setiosflags(ios::fixed) << f1 << ' ' << f2 << endl;

    /* 第2组 */
    cout << endl;
    cout << setiosflags(ios::scientific) << f1 << ' ' << f2 << endl;

    return 0;
}
```

运行截图：




```
#include <iostream>
#include <iomanip>

using namespace std;
int main()
{
    float f1 = 1234.5678F, f2 = 8765.4321F;

    /* 第1组 */
    cout << f1 << ' ' << f2 << endl;
    cout << setiosflags(ios::scientific) << f1 << ' ' << f2 << endl;

    /* 第2组 */
    cout << endl;
    cout << setiosflags(ios::fixed) << f1 << ' ' << f2 << endl;

    return 0;
}
```

运行截图：



§.基础知识题 – C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

I.ios::fixed和ios::scientific的混合使用 – 在上一页的基础上将程序改正确，并给出截图

```
#include <iostream>
#include <iomanip>

using namespace std;
int main()
{
    float f1 = 1234.5678F, f2 = 8765.4321F;

    cout << f1 << ' ' << f2 << endl;
    cout << setiosflags(ios::fixed) << f1 << ' ' << f2 << endl;

    cout << resetiosflags(ios::fixed);

    cout << endl;
    cout << setiosflags(ios::scientific) << f1 << ' ' << f2 << endl;

    return 0;
}
```

```
1234.57 8765.43
1234.567749 8765.431641

1.234568e+03 8.765432e+03
```

```
#include <iostream>
#include <iomanip>

using namespace std;
int main()
{
    float f1 = 1234.5678F, f2 = 8765.4321F;

    cout << f1 << ' ' << f2 << endl;
    cout << setiosflags(ios::scientific) << f1 << ' ' << f2 << endl;

    cout << resetiosflags(ios::scientific);

    cout << endl;
    cout << setiosflags(ios::fixed) << f1 << ' ' << f2 << endl;

    return 0;
}
```

```
1234.57 8765.43
1.234568e+03 8.765432e+03

1234.567749 8765.431641
```

结论：（再强调一遍，先去读P.5，后续不再提示）

如果想要在一个程序中同时显示fixed和scientific形式，需要在两者之间加入一句：

_____cout << resetiosflags(*);_____

§.基础知识题 – C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

J.setw的基本使用 – (1)

```
#include <iostream>
#include <iomanip>

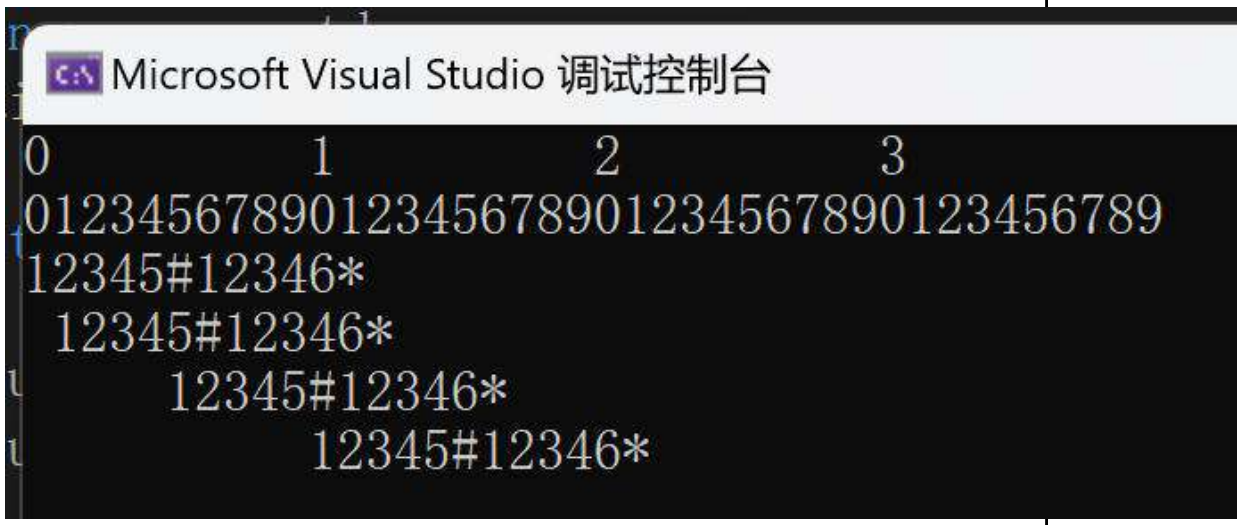
using namespace std;
int main()
{
    int a = 12345;

    cout << "0         1         2         3" << endl;
    cout << "01234567890123456789012345678901234567890123456789" << endl;

    cout << setw(3) << a << '#' << a + 1 << '*' << endl;
    cout << setw(6) << a << '#' << a + 1 << '*' << endl;
    cout << setw(10) << a << '#' << a + 1 << '*' << endl;
    cout << setw(15) << a << '#' << a + 1 << '*' << endl;

    return 0;
}
```

运行截图：



结论：

1、setw指定的宽度是总宽度，当总宽度大于数据宽度时，显示规律为：在数据前面补空格，算上空格后宽度为setw指定的宽度；当总宽度小于数据宽度时，显示规律为 _输出完整数据，不补也不删_____

2、setw的设置后，对后面的___仅一个_____(仅一个/所有)数据有效

3、程序最前面两行的输出，目的是什么？：为了让后面输出数据的宽度看起来更直观

4、每行输出的最后一个*，目的是什么？：为了确定每个数据结束的地方，以便计算宽度

§.基础知识题 - C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

J.setw的基本使用 - (2)

```
#include <iostream>
#include <iomanip>

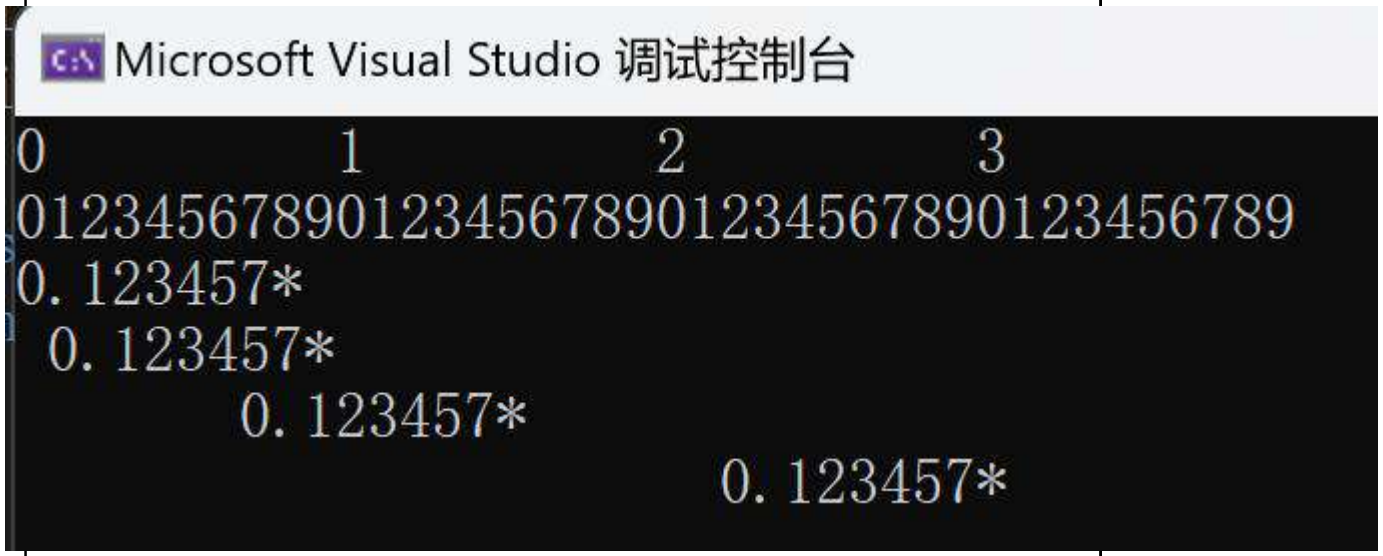
using namespace std;
int main()
{
    double a = 0.123456789012345;

    cout << "0         1         2         3" << endl;
    cout << "01234567890123456789012345678901234567890123456789" << endl;

    cout << setw(6) << a << '*' << endl;
    cout << setw(9) << a << '*' << endl;
    cout << setw(15) << a << '*' << endl;
    cout << setw(30) << a << '*' << endl;

    return 0;
}
```

运行截图：



结论：

1、setw指定的宽度是总宽度，对于实型数据，_包含____(包含/不包含)小数点

§.基础知识题 - C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

K.setw+setfill的使用

```
#include <iostream>
#include <iomanip>

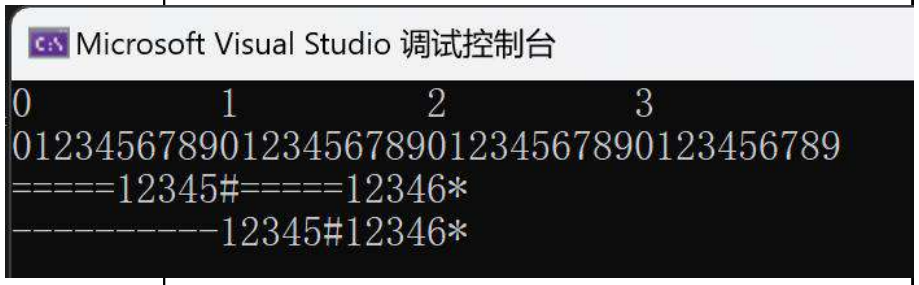
using namespace std;
int main()
{
    int a = 12345;

    cout << "0         1         2         3" << endl;
    cout << "01234567890123456789012345678901234567890123456789" << endl;

    cout << setfill('=') << setw(10) << a << '#' << setw(10) << a + 1 << '*' << endl;
    cout << setw(15) << setfill('-') << a << '#' << a + 1 << '*' << endl;

    return 0;
}
```

运行截图：



结论：

1、setfill的作用是___设置setw的填充字符_____

2、setfill的设置后，对后面的__所有_____(仅一个/所有)数据有效

3、解释为什么第4行的第2个数(12346)前面没有-：因为setw设置后对后面的仅一个数据有效

§.基础知识题 – C++方式输入输出的格式化控制

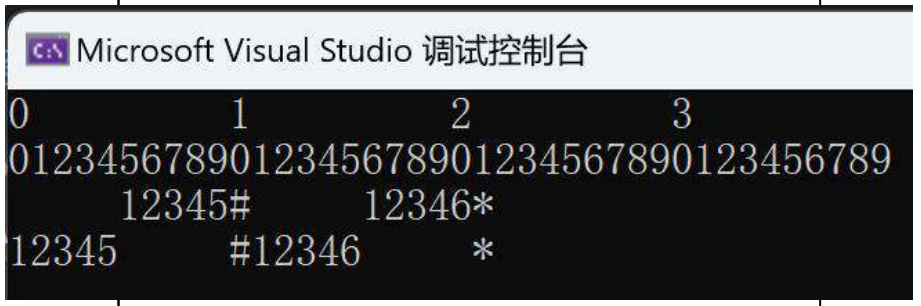
1、在cout中使用格式化控制符

L.setw/setfill与ios::left/ios::right的混合使用 – (1)

```
#include <iostream>
#include <iomanip>

using namespace std;
int main()
{
    int a = 12345;
    cout << "0         1         2         3" << endl;
    cout << "01234567890123456789012345678901234567890123456789" << endl;
    cout << setw(10) << a << '#' << setw(10) << a + 1 << '*' << endl;
    cout << setiosflags(ios::left);
    cout << setw(10) << a << '#' << setw(10) << a + 1 << '*' << endl;
    return 0;
}
```

运行截图：




```
#include <iostream>
#include <iomanip>

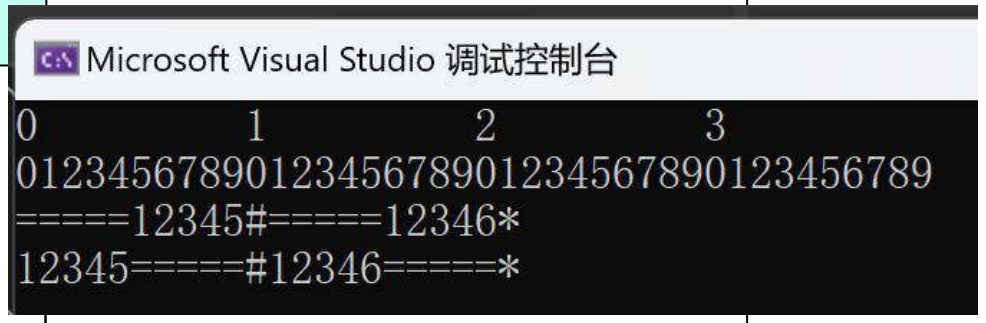
using namespace std;
int main()
{
    int a = 12345;
    cout << "0         1         2         3" << endl;
    cout << "01234567890123456789012345678901234567890123456789" << endl;
    cout << setfill('=') << setw(10) << a << '#' << setw(10) << a + 1 << '*' << endl;
    cout << setiosflags(ios::left);
    cout << setfill('=') << setw(10) << a << '#' << setw(10) << a + 1 << '*' << endl;
    return 0;
}
```

结论：

1、ios::left的作用是__使输出的数据左对齐__

2、如果不设置，缺省是___右____(左/右对齐)

运行截图：



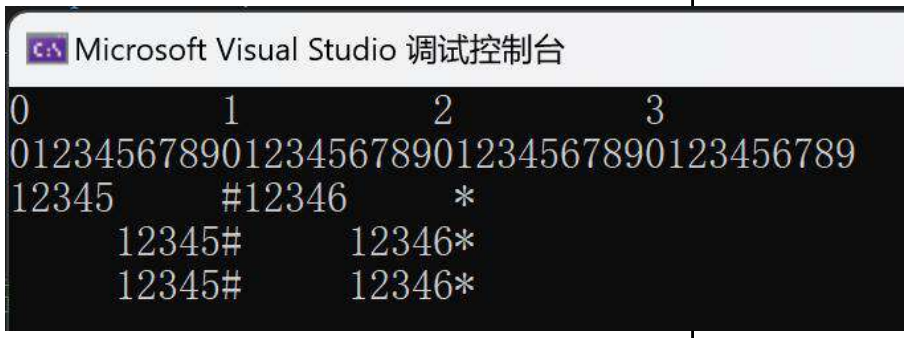
§.基础知识题 - C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

L.setw/setfill与ios::left/ios::right的混合使用 - (2) - 同时使用(错误)

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    int a = 12345;
    cout << "0         1         2         3" << endl;
    cout << "01234567890123456789012345678901234567890123456789" << endl;
    /* 左对齐 */
    cout << setiosflags(ios::left) << setw(10) << a << '#' << setw(10) << a + 1 << '*' << endl;
    /* 右对齐 */
    cout << setiosflags(ios::right) << setw(10) << a << '#' << setw(10) << a + 1 << '*' << endl;
    /* 左对齐 */
    cout << setiosflags(ios::left) << setw(10) << a << '#' << setw(10) << a + 1 << '*' << endl;
    return 0;
}
```

运行截图：




```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    int a = 12345;
    cout << "0         1         2         3" << endl;
    cout << "01234567890123456789012345678901234567890123456789" << endl;
    /* 右对齐 */
    cout << setiosflags(ios::right) << setw(10) << a << '#' << setw(10) << a + 1 << '*' << endl;
    /* 左对齐 */
    cout << setiosflags(ios::left) << setw(10) << a << '#' << setw(10) << a + 1 << '*' << endl;
    return 0;
}
```

运行截图：



§.基础知识题 – C++方式输入输出的格式化控制

1、在cout中使用格式化控制符

L.setw/setfill与ios::left/ios::right的混合使用 – 在上一页的基础上将程序改正确，并给出截图

week3 (全局范围) main()

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    int a = 12345;
    cout << "0         1         2         3" << endl;
    cout << "01234567890123456789012345678901234567890123456789" << endl;
    /* 左对齐 */
    cout << setiosflags(ios::left) << setw(10) << a << '#' << setw(10) << a + 1 << '*' << endl;
    /* 右对齐 */
    cout << setiosflags(ios::right) << setw(10) << a << '#' << setw(10) << a + 1 << '*' << endl;
    /* 左对齐 */
    cout << resetiosflags(ios::right);
    cout << setiosflags(ios::left) << setw(10) << a << '#' << setw(10) << a + 1 << '*' << endl;
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
0         1         2         3
01234567890123456789012345678901234567890123456789
12345     #12346    *
     12345#    12346*
12345     #12346    *
```

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    int a =
    cout <
    cout <
    /* 右对
    cout <
    /* 左对
    cout <
    return
}
```

结论：

如果想要right对齐后再left对齐，需要在两者之间加入一句：

_____cout << resetiosflags(ios::right);_____

运行截图：

week3 (全局范围) main()

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    int a = 12345;
    cout << "0         1         2         3" << endl;
    cout << "01234567890123456789012345678901234567890123456789" << endl;
    /* 右对齐 */
    cout << setiosflags(ios::right) << setw(10) << a << '#' << setw(10) << a + 1 << '*' << endl;
    /* 左对齐 */
    cout << resetiosflags(ios::right);
    cout << setiosflags(ios::left) << setw(10) << a << '#' << setw(10) << a + 1 << '*' << endl;
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
0         1         2         3
01234567890123456789012345678901234567890123456789
     12345#    12346*
12345     #12346    *
```

§.基础知识题 - C++方式输入输出的格式化控制



此页不要删除，也没有意义，仅仅为了分隔题目

§.基础知识题 - C++方式输入输出的格式化控制

2、在cin中使用格式化控制符

A.基本要求：从键盘输入16进制数

```
#include <iostream>
#include <iomanip>

using namespace std;
int main()
{
    short a;
    cin >> hex >> a;

    cout << "dec:" << dec << a << endl;
    cout << "hex:" << hex << a << endl;
    cout << "oct:" << oct << a << endl;

    return 0;
}
```

1、输入：1a2b↙ （合理正数）



2、输入：a1b2↙ （超上限但未超同类型的unsigned上限）



3、输入：fffff↙ （超上限且超过同类型的unsigned上限）



4、输入：-1a2b↙（合理负数）



5、输入：-fffff↙（超下限）



§.基础知识题 – C++方式输入输出的格式化控制

2、在cin中使用格式化控制符

B.基本要求：从键盘输入8进制数（自行构造测试数据）

```
#include <iostream>
#include <iomanip>

using namespace std;
int main()
{
    int a;
    cin >> setbase(8) >> a;

    cout << "dec:" << dec << a << endl;
    cout << "hex:" << hex << a << endl;
    cout << "oct:" << oct << a << endl;

    return 0;
}
```



1、输入：__123__↙ （合理正数）



2、输入：_ 30653520525 ___ ↙ （超上限但未超同类型的unsigned上限）



3、输入：__ 112402761777__ ↙ （超上限且超过同类型的unsigned上限）



4、输入：__-123__ ↙（合理负数）

5、输入：_-112402761777 ___↙（超下限）



§.基础知识题 – C++方式输入输出的格式化控制

2、在cin中使用格式化控制符

C.格式控制符setiosflags(ios::skipws)的使用

```
#include <iostream>
using namespace std;

int main()
{
    int a, b;

    cin >> a >> b;

    cout << a << endl;
    cout << b << endl;
    return 0
}
```

Microsoft Visual Studio 调试控制台

```
12 34
12
34
```

假设键盘输入为：12 34↙

则输出为：_12 ↙34 ↙ __

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    int a, b;
    cin >> setiosflags(ios::skipws);
    cin >> a >> b;
    cout << a << endl;
    cout << b << endl;
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
12 34
12
34
```

假设键盘输入为：12 34↙

则输出为：_ 12 ↙34 ↙ ____

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    int a, b;
    cin.unsetf(ios::skipws);
    cin >> a >> b;
    cout << a << endl;
    cout << b << endl;
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
12 34
12
0
```

假设键盘输入为：12 34↙

则输出为：_ 12 ↙0 ↙ ____

综合以上三个例子可以得到如下结论：

1、“忽略前导空格”的意思，是空格不作为_0_____，而是做为____使输入终止（因此导致第3个例子b未取得34）

2、setiosflags(ios::skipws)在缺省情况下是__有效___(有效/无效)的，即不设置也生效

3、如果想取消"忽略前导空格"的设置，应使用_____ cin.unsetf(ios::skipws) _____

§.基础知识题 - C++方式输入输出的格式化控制



此页不要删除，也没有意义，仅仅为了分隔题目